Precog Task Report: The Lazy Artist

Objective:

The objective of this project is to investigate shortcut learning in neural networks using a color-biased version of MNIST. We aim to:

- Design a biased dataset
- Train a SimpleCNN in this biased dataset and record their train and test accuracy.
- Probe into the early and deeper layer of SimpleCNN
- Analyze model behavior using Grad-CAM.
- Prepare a robust model to mitigate bias issues.
- Evaluate adversarial vulnerability via targeted PGD attacks.Compare baseline and robust models quantitatively.
- Exploring Sparse Autoencoder.

All the tasks from 0 to 6 have been executed.

Task 0: The Biased Canvas(Dataset)

Introduction:

MNIST dataset is a dataset which is used for benchmark of Machine Learning models. It is regarded as the “HELLO WORLD” of Computer Vision.

Key properties:

- Structure: 60,000 training, 10,000 testing images.
- Format: 28×28 pixel grayscale images of digits 0-9.
- Usage: Used for classification, computer vision research, and testing algorithms.

Execution:

MNIST needs to be modified in a way that 95% images of a single digit will be incorporated with a particular color. For example 0 is coded to Red. The remaining 5% can be any other color. This is repeated for all the digits. This is done to the Training Set of the MNIST dataset. The Testing Set is also modified where the correlation is inverted i.e. 0s are never red. The grayscale is converted to RGB. A fixed random seed is used to ensure the same color assignments and the same biased distribution during every run. The color is applied to the foreground stroke i.e. digits are colored. The intention is to create a shortcut signal where color strongly predicts label.

Key Properties:

- 95% images of single digits of the Training Set are assigned a single color. For eg. 0s are red.

- Testing set colors has been randomised and it is made sure that 0s are never Red here.

Fig 1. The Modified Dataset.

Verification of the 95% criteria has been done as well.

```
Digit | Dominant Color %
-----------------------
    0 | 0.9480
    1 | 0.9506
    2 | 0.9554
    3 | 0.9517
    4 | 0.9451
    5 | 0.9478
    6 | 0.9534
    7 | 0.9512
    8 | 0.9511
    9 | 0.9509
```

Fig 2. The verification of dataset

Task 1: The Cheater (SimpleCNN):

Objective:

A standard simple 3 layer CNN is to be trained on the biased dataset created earlier naturally and the observation to be noted.

Introduction:

Convolutional Neural Networks (CNNs) are deep learning models designed to automatically detect and classify patterns in visual data like images and videos. By using convolutional, pooling, and fully connected layers, CNNs extract hierarchical features—from simple edges to complex shapes—making them the standard for computer vision tasks such as object detection, facial recognition, and medical imaging.

CNN Design:

A simple 3 layer CNN has been chosen for this task.

The design of the CNN is:

Block 1

- Conv2D: 3 → 32 channels
- Kernel size: 3×3
- Padding: 1 (preserves spatial resolution)
- Activation: ReLU
- MaxPooling: 2×2

Block 2

- Conv2D: 32 → 64 channels
- Kernel: 3×3
- Activation: ReLU
- MaxPooling: 2×2

Block 3

- Conv2D: 64 → 128 channels
- Kernel: 3×3
- Activation: ReLU
- Adaptive Average Pooling → Output size: 1×1

Loss Function: Cross Entropy Loss was used as it is multi class classification. MSE is used for regression. BCE is for binary class.

Execution:

The Simple CNN has been trained and tested on the modified MNIST dataset. It has been done in 2 versions and i.e. for 5

epochs and 10 epochs. It has been chosen for experimental and curiosity purposes. The Training set has been divided into 85% for training and 15% for validation.

Results (Epoch 5):

The results after training the Simple CNN for 5 epochs as shown below:

```
Epoch 1: Train Acc = 0.9496, Val Acc = 0.9524
Epoch 2: Train Acc = 0.9508, Val Acc = 0.9536
Epoch 3: Train Acc = 0.9514, Val Acc = 0.9537
Epoch 4: Train Acc = 0.9513, Val Acc = 0.9540
Epoch 5: Train Acc = 0.9580, Val Acc = 0.9610
```

Fig 3. Training and validation accuracy

The result shown above is within expectation i.e. the training and validation accuracy is beyond 95% after training on the Biased Easy Train Set.

```
Hard Test Accuracy = 0.1803
```

Fig 4. Testing accuracy

The accuracy shown above after evaluating the model on the Hard Test set is also within expectation i.e. below 20%.

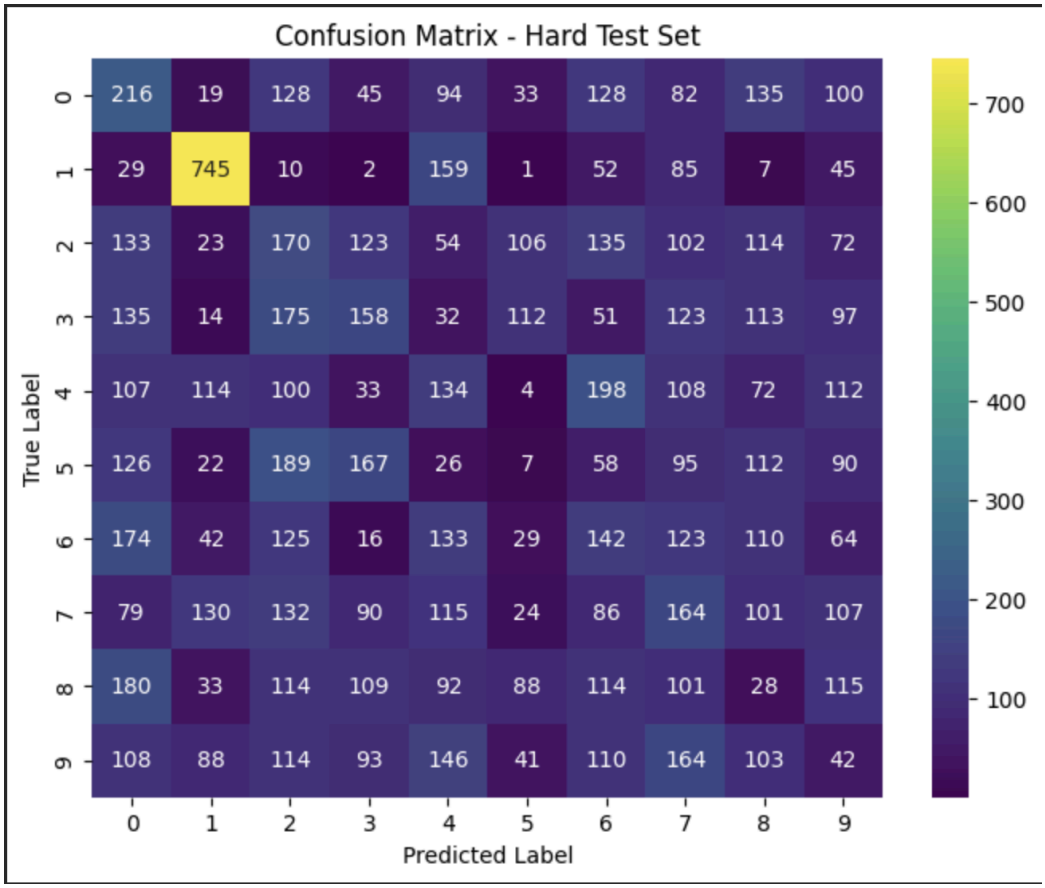

Fig 5. Confusion Matrix

The confusion matrix on the Hard Test Set shows that except for digit 1, predictions for all other digits are highly scattered across many classes. This proves that it failed to generalize.

Category	Count
Color-based prediction	5
Correct(shape based)	4
Neither	1
Total samples	10

Table 1: Conflict Case

10 conflict samples were evaluated from the hard test set, where the digit label and assigned color do not match. Each prediction was categorized as:

- **Color-based prediction**: Model prediction matches the color index.
- **Correct (Shape-based)**: Model prediction matches the true digit.
- **Neither / Random**: Prediction matches neither the digit nor the color.

It can be seen 50% were color based, 40% were shape based and remaining 10% percent were neither of other parameters.

Results (Epoch 10):

The results after training the Simple CNN for 10 epochs as shown below:

```
Epoch 1: Train Acc = 0.9496, Val Acc = 0.9524
Epoch 2: Train Acc = 0.9507, Val Acc = 0.9534
Epoch 3: Train Acc = 0.9512, Val Acc = 0.9538
Epoch 4: Train Acc = 0.9538, Val Acc = 0.9560
Epoch 5: Train Acc = 0.9578, Val Acc = 0.9599
Epoch 6: Train Acc = 0.9622, Val Acc = 0.9630
Epoch 7: Train Acc = 0.9702, Val Acc = 0.9717
Epoch 8: Train Acc = 0.9722, Val Acc = 0.9741
Epoch 9: Train Acc = 0.9785, Val Acc = 0.9790
Epoch 10: Train Acc = 0.9789, Val Acc = 0.9791
```

Fig 6. Training and validation accuracy

The result shown for 10 above is within expectation as well i.e. the training and validation accuracy is beyond 95% after training on the Biased Easy Train Set.

```
Hard Test Accuracy = 0.6340
```

Fig 7. Testing accuracy

Surprisingly, here the Hard Test Accuracy went beyond 60%. We can interpret that running for 10 epochs, the model starts to generalise.

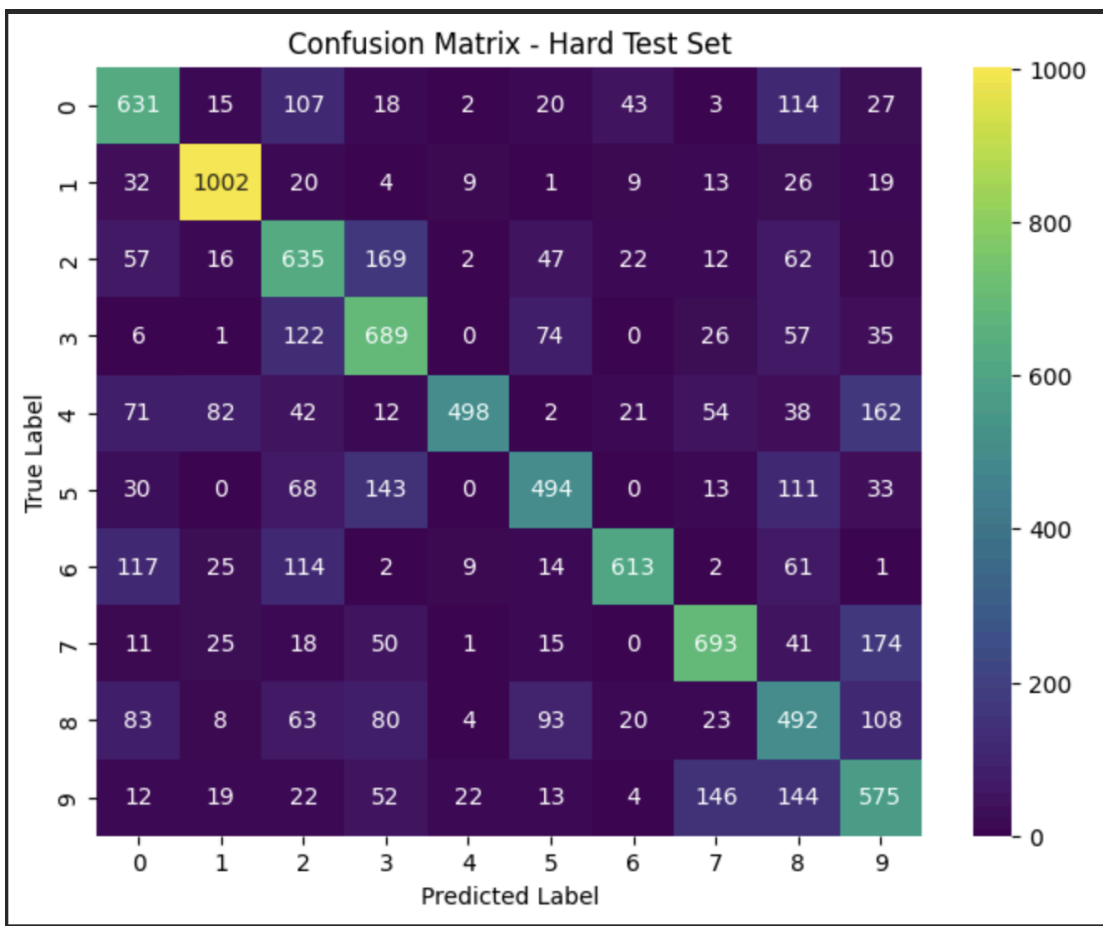

Fig 8. Confusion Matrix

The confusion matrix on the Hard Test Set for 10 epochs shows that all the digits were respectfully predicted. Digit 1 was particularly strong at 1002. This shows that the model has started to generalise by a good margin.

Category	Count
Color-based prediction	3
Correct(shape based)	5
Neither	2
Total samples	10

Table 2: Conflict Case

It can be seen that 50% were correct(shape) based, 30% were color based and the remaining 20% percent were neither. It can be interpreted that running for more iterations/epoch can help the model generalise better.

Task 2: The Prober

Objective:

To look into internally what features Simple CNN learning during Early layer or Block 1 and Deep layer or Block 3.

Execution:

In order to probe, activation maximization was performed. A random input image was iteratively optimised using gradient ascent of a specific convolutional filter. This reveals the type of pattern that strongly activates a neuron. This was done in Early Layers or Block 1 and Deeper layer or Block 3.

Results (Epoch 5):

Early Layers: The first convolutional layer has 32 filters which are from 0 to 31 from which 0,5,10,15,20 and 25 were chosen.

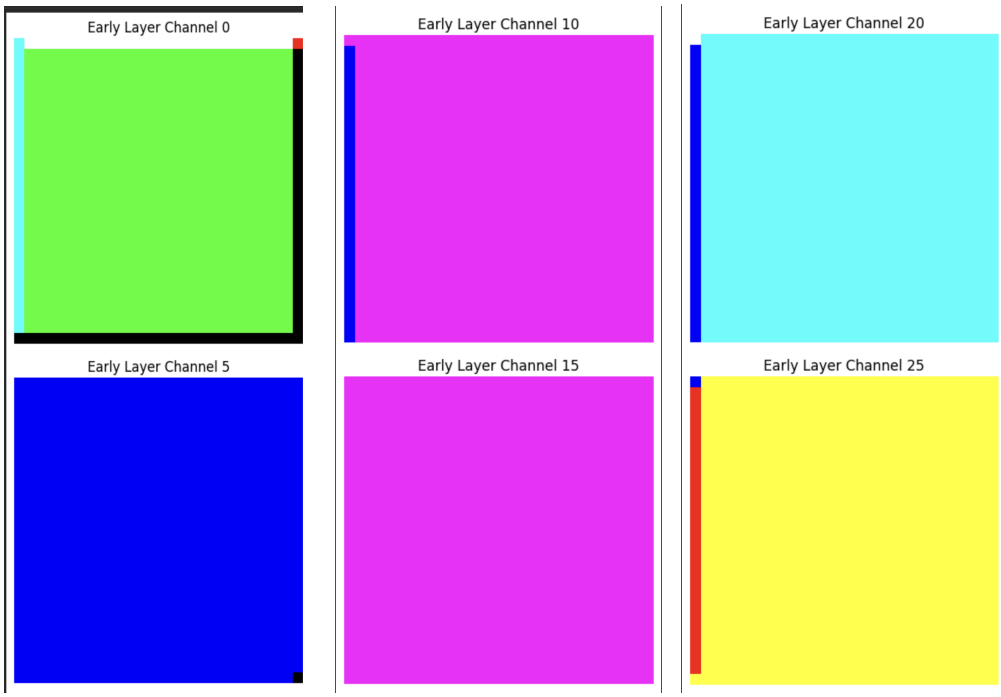

Fig 9: The "vision" neurons sees

This output signifies that in early layers, huge reliance on color is present.

Deeper Layers: The third convolutional layer has 128 filters which are from 0 to 127 from which 0,10, 25 , 50, 75 and 100 were chosen.

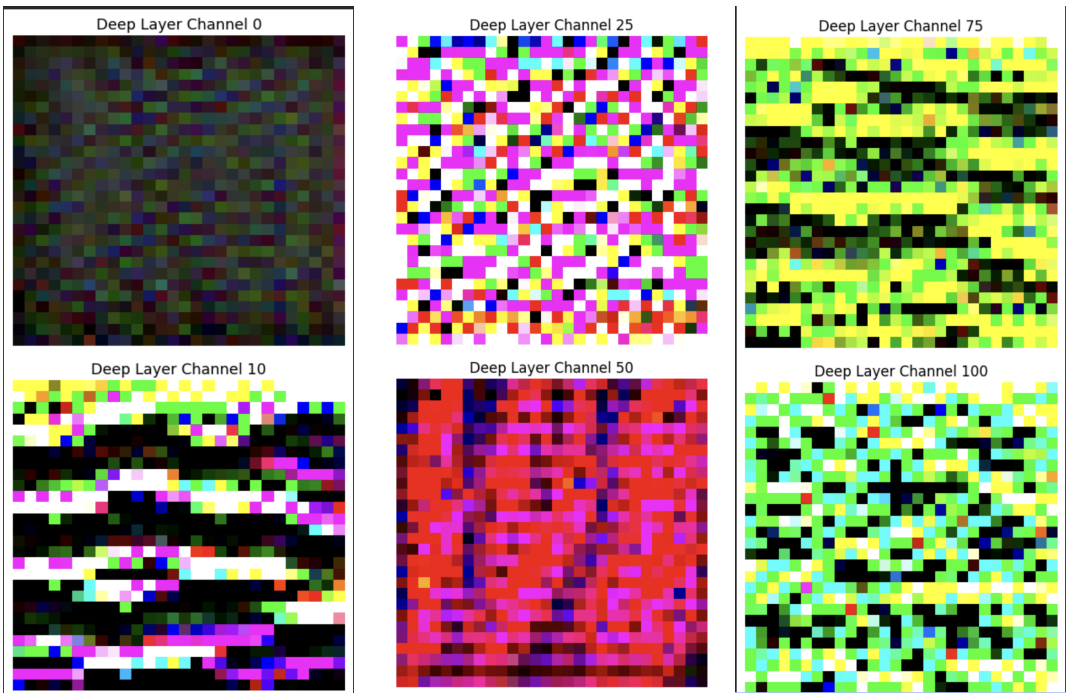

Fig 10: The "vision" neurons sees

The output signifies that some patterns like horizontal bands are visible but still strong color bias is present. There is limited evidence of stroke based representations.

<u>Results (Epoch 10):</u>

Early Layers: The first convolutional layer has 32 filters which are from 0 to 31 from which 0,5,10,15,20 and 25 were chosen.

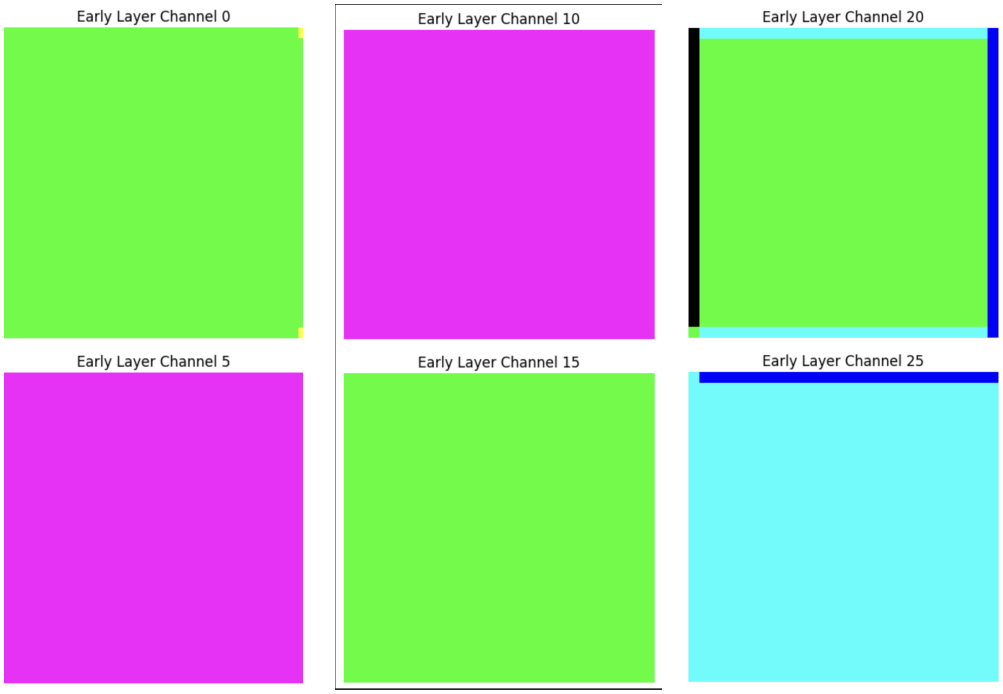

Fig 9: The “vision” neurons sees

This output from a model ran for 10 epoch signifies the same as in the 5 epoch ran model and that is in early layers, huge reliance on color is present. No spatial pattern detection.

Deeper Layers: The third convolutional layer has 128 filters which are from 0 to 127 from which 0,10, 25 , 50, 75 and 100 were chosen.

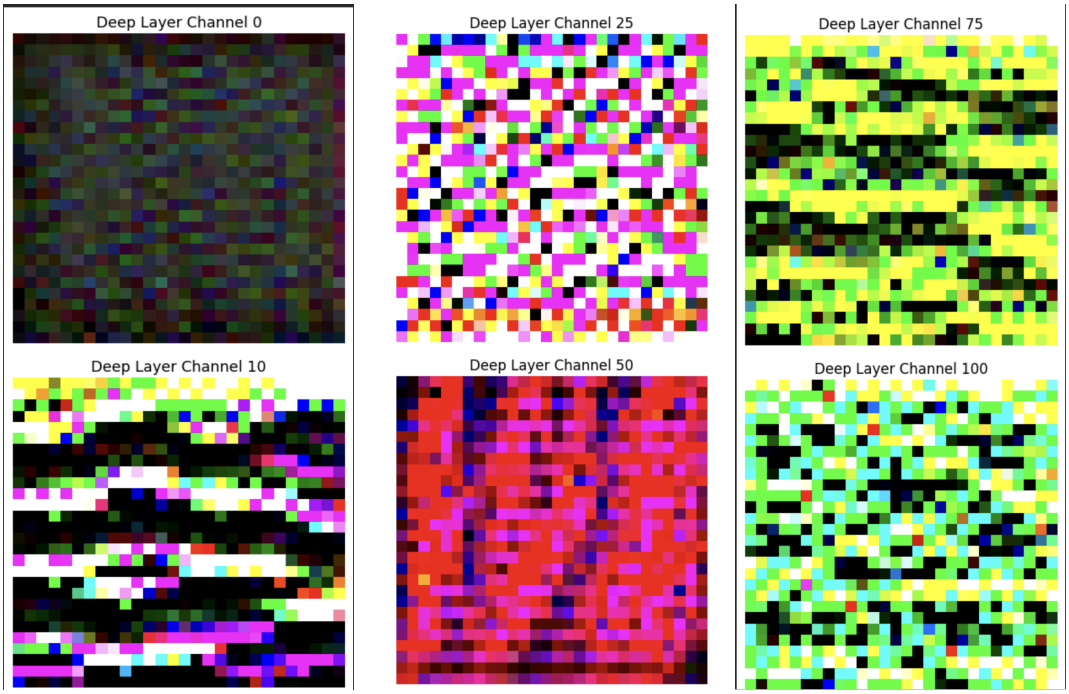

Fig 10: The “vision” neurons sees

The output signifies almost the same patterns as the model ran for 5 epochs i.e. some patterns like horizontal bands are visible but still strong color bias is present. There is limited evidence of stroke based representations. It still didn’t transition to shape-dominant representations.

Task 3: The interrogation

Objective:

To mathematically verify whether the model relies on shape of the digit or color by visualizing spatial attention using Grad-CAM implemented from Scratch

Execution and Mathematical Description:

Grad-CAM(Gradient-weighted Class Activation Mapping) is used in deep learning and computer vision to generate heatmaps for decisions made by the Convolutional Neural Networks.

It works by:

- Extracting feature maps from the last convolutional layer
- Computing gradients of the predicted class score
- Averaging gradients across spatial dimensions
- Weighting feature maps by these gradients
- Applying ReLU and normalization

Mathematical Formulation:

$\alpha^{c} = (1 / Z) \Sigma_i \Sigma_j (\partial y^c / \partial A^k_{ij})$

where

- $Z = H \times W$ is the total number of spatial locations.
- the gradient of the class score with respect to spatial activation (i,j)

Grad-CAM Heatmap

$L^c_GradCAM = ReLU (\Sigma\square \ \alpha\square^c \ A^k)$

<u>Results (Epoch 5):</u>

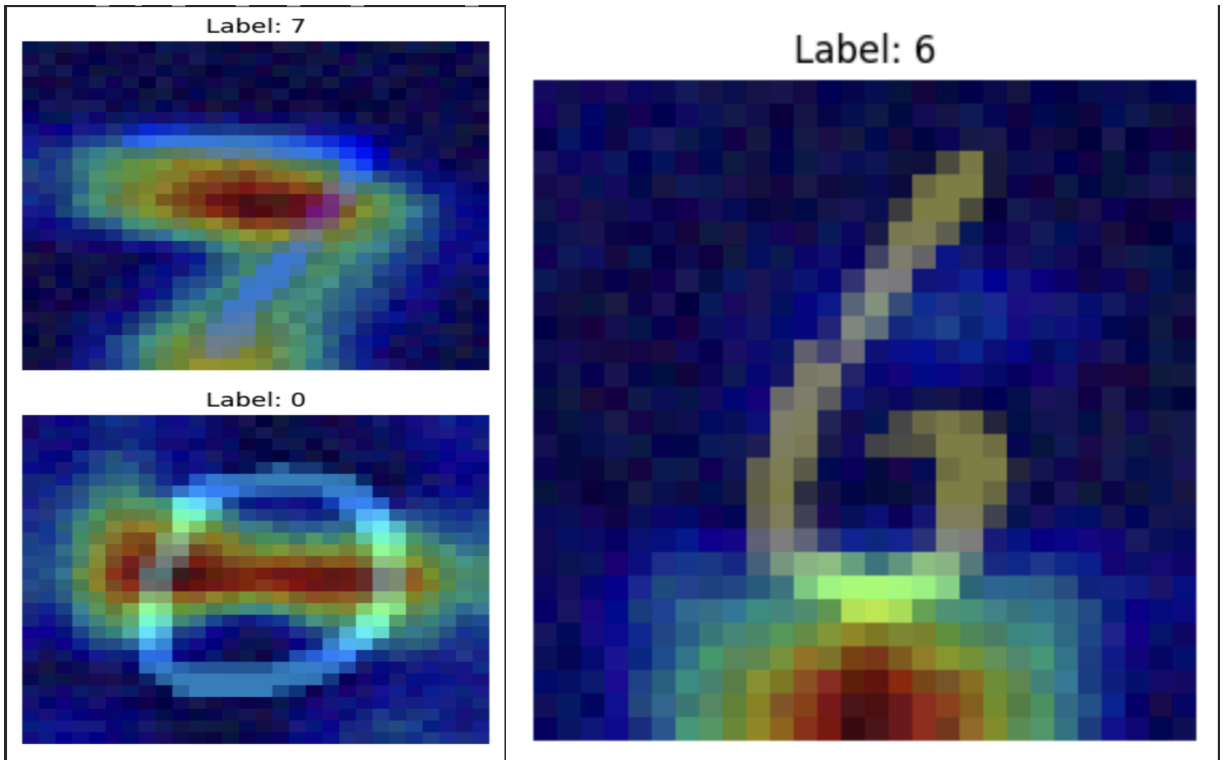

Fig 11: The heatmap by Grad-CAM for 5 epochs

The heatmap by Grad-CAM of the 5-epoch model reveals that the network does not sharply focus on digits strokes. It partially captures it. It mostly relying on color shortcuts rather than structural understanding. Label 7 and Label 0 are mostly likely to have picked up some structural shapes whereas for

Label 6 it is at the bottom. Color gradient is strongest at bottom in this case so color bias is strong here.

Results (Epoch 10):

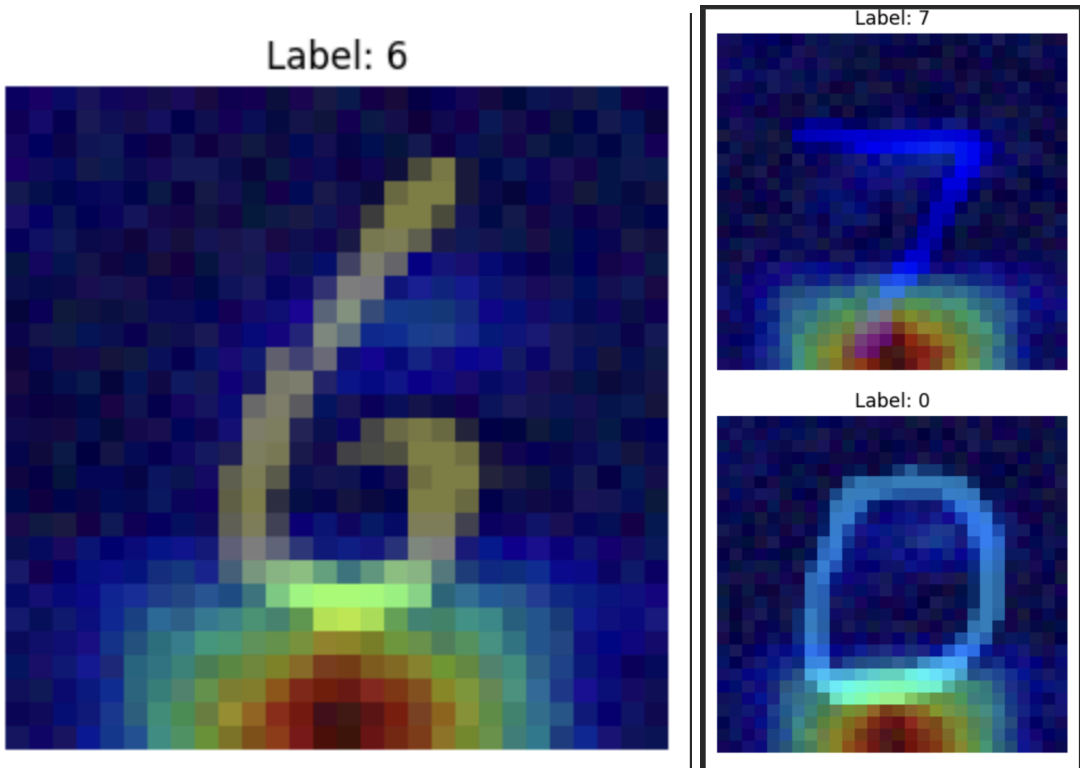

Fig 12: The heatmap by Grad-CAM for 10 epochs

The behavior exhibited by the model ran for 10 epochs is strange as the heat map is concentrating for all labels. Most likely after the predictive signal is highest there and some new shortcut feature has been learnt rather than relying on the structure.

AHA Moment:

Biased example: Color matches Digit

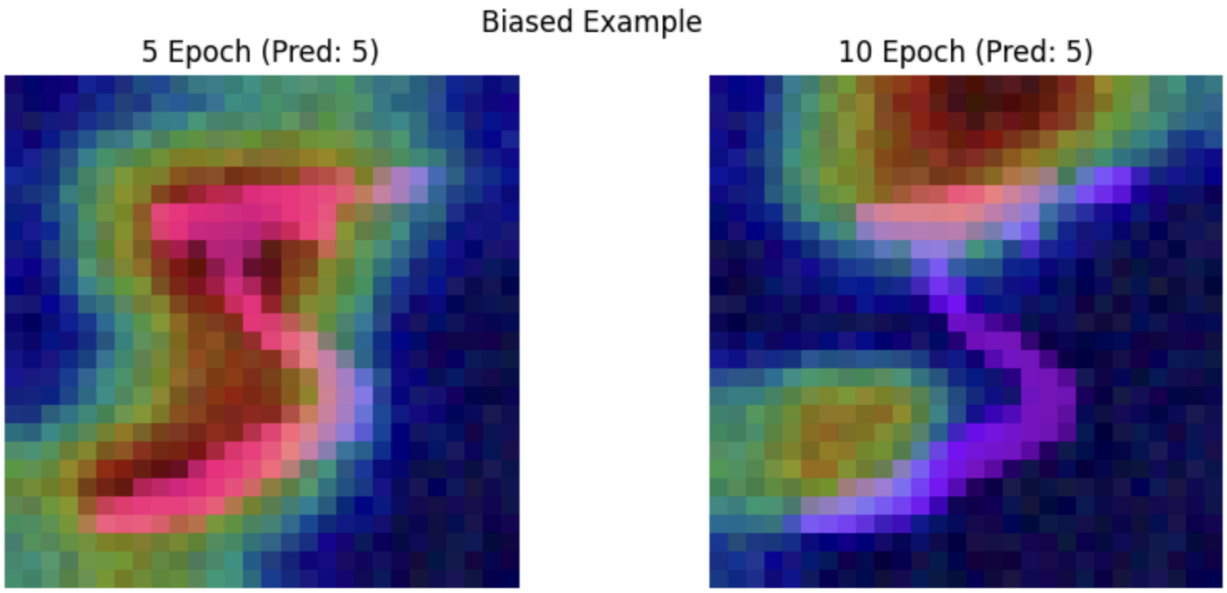

Fig 13. Biased Examples

For biased samples, where the digit and color are strongly correlated (e.g., Red 0), the Grad-CAM heatmaps in the 5-epoch model shows that heatmap is wide over the colored region of the digit. The activation does not isolate the structure sharply but instead spreads across the entire colored foreground. This indicates that the model primarily relies on color cues rather than digit shape.In the 10-epoch model, the activation becomes slightly more structured, with somewhat clearer emphasis on digit strokes. However,

color-driven attention is still prominent. This suggests minor improvement in shape awareness, but the bias is still there.

Conflict Examples: Color contradicts digits

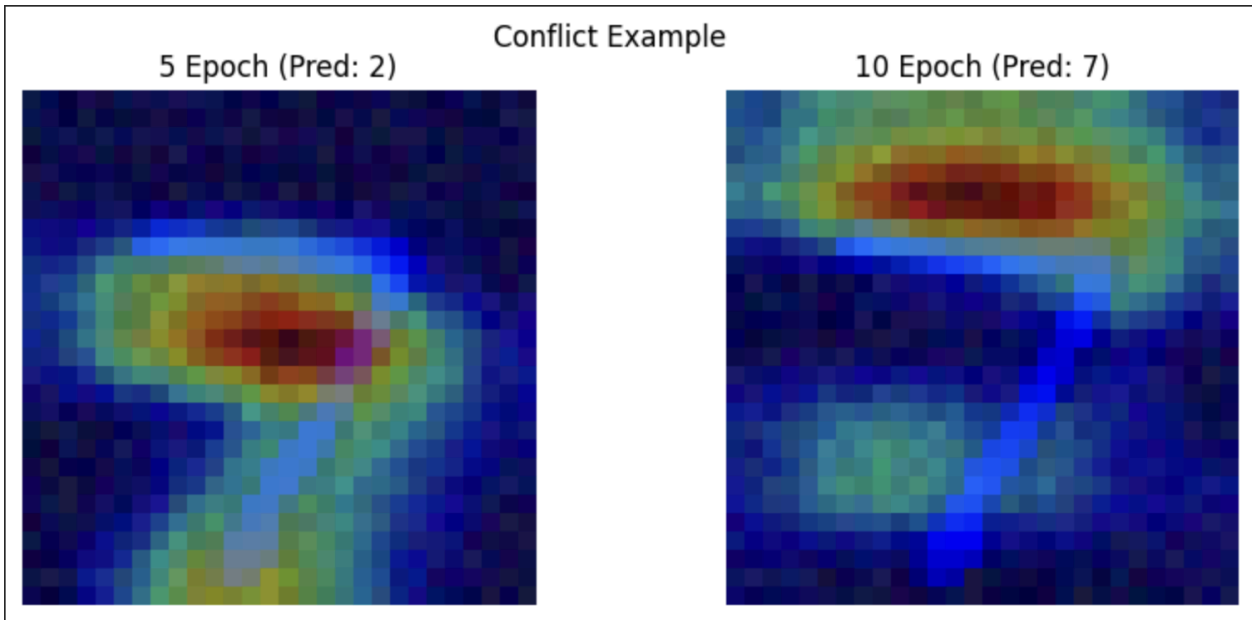

Fig 14. Conflict examples

For conflict samples (e.g., Green 0), where color does not match the training bias, the 5-epoch model's heatmap strongly concentrates on the upper colored region and often predicts the class associated with that color instead of the true digit. This confirms shortcut learning — the model prioritizes color over shape.In the 10-epoch model, the heatmap shows comparatively more attention on digit contours in some cases, and predictions are occasionally corrected. However, significant activation still appears over the colored regions, indicating that the learned bias has not been fully removed.

Task 4: The Intervention

Objective:

To retrain the model to ignore color and focus on the shape without converting to grayscale and without changing the dataset(i.e. the 95% bias) and achieve accuracy above 70% on the Hard Test Set.

Version 1: Sobel Filter

Sobel filter is an edge filter used for preprocessing steps to encourage the model to learn the shape(edges and gradients) rather than color. Sobel filter extracts intensity gradients which means shape is prioritized over color.
Sobel edge filter uses two filters in two directions. They are:

```
Gx =[ [-1, 0, 1],
      [-2, 0, 2],
      [-1, 0, 1]]
Gy = [[-1, -2, -1],
      [ 0,  0,  0],
      [ 1,  2,  1]]
```

Gradient Calculation = (Gx^2 + Gy^2)^½
For each input image:

- The original colored image was passed through the model.
- A Sobel edge-transformed version of the same image was generated.

- Both versions were forwarded through the network.
- A consistency constraint was applied between their predictions.

This forces the model to produce similar predictions for:

- The full-color image
- Its edge-only structural representation

The total training loss was defined as:

Loss Function = CrossEntropy + lambda.MSE(p, p_{edge})

where:

- P is normal image and Pedge is sobel filter version.
- CrossEntropy ensures correct digit classification.
- MSE enforces prediction consistency between original and edge images.
- lambda controls the strength of structural regularization.

Result:

Although the method looked promising, but even after running for 10 epochs, the Hard Test Set accuracy didn't reach the target. The Hard Test Accuracy is 60.3%. Due to this poor performance, this was kept as a result of experimental idea and hence the 5 epoch version wasn't kept.

Version 2: Color Randomization + Channel Dropout

Objective:

In order to reduce Model's dependency on biased color-digit relation, Two interventions were used. They are random color

perturbations and randomly suppressing color channels during training.

Execution:

1) **Strong Color Jitter:**
   A ColorJitter transformation was applied

- Brightness = 0.6
- Contrast = 0.6
- Saturation = 0.6
- Hue = 0.3

This causes the training images to undergo random color transformation. In other words, if color keeps changing then the model can not trust it as a stable signal.

2) **Channel Dropout:**

With probability 0.3, a randomly selected RGB channel was zeroed out during training. It prevents the model from heavily relying on a single dominant color channel.

Loss Function = CrossEntropy Loss.

Result: This method performed admirably and crossed the target set as evident in the table below.

Number of epochs	Hard Test accuracy
5	78.67%
10	87.9 %

Table 3: 5 and 10 epoch accuracies

Version 2: Color Penalty

Objective:

To enforce prediction invariance under color perturbations by introducing a consistency penalty between original and color-shifted predictions.

Execution:

1) Color Perturbation Branch

A ColorJitter transformation was applied to generate a color-shifted version of each input image:

- Brightness = 0.5
- Contrast = 0.5
- Saturation = 0.7
- Hue = 0.3

Both the original image and its color-perturbed version were passed through the network.

2) Color Invariance Loss

The probability distributions of both predictions were aligned using MSE:

Loss = CrossEntropy + lambda.MSE(p,p_{color})

where:

- P is a normal image and Pcolor is the color-perturbed version.
- CrossEntropy ensures correct digit classification.
- MSE penalizes prediction differences under color changes.

- lambda = 0.7 controls the strength of invariance enforcement.

Result:

The model performed poorly in early training (5 epochs), but significantly improved after longer running i.e. 10 epochs. This indicates that explicit invariance regularization slows early convergence but can improve robustness with sufficient training.

Number of epochs	Hard Test accuracy
5	28.58 %
10	73.94 %

Table 4: 5 and 10 epoch accuracies

Conclusion:

This experimental story was a success. It started with an accuracy of **60.3%** in **Version 1** to **88%** in **Version 2** and ultimately **73.8%** in **Version 3** (all these cases for 10 epoch). The target was reached in **2 out of 3 cases.**

Task 5: The Invisible Cloak

Objective:

To evaluate the robustness of trained models using adversarial attack. The attack attempts to modify the input image slightly so that the model misclassifies a digit 7 as digit 3 while keeping the perturbation visually imperceptible. This is done on both robust model (task 4) and lazy model (task 1) and the difference is quantified.

Introduction:

The attack has been done by Projected Gradient Descent. PGD is an iterative adversarial attack that modifies the input image by following the gradient of loss function. At each iteration, the image is perturbed in a direction that increases the likelihood of the target class.

Update rule:

$$x_{(t+1)} = x_t - \alpha \cdot \text{sign}(\nabla_x L(f(x_t), y_{target}))$$

where:

- **x_t** = adversarial image at iteration *t*
- **α** = step size (attack learning rate)
- **L** = CrossEntropy loss
- **y_{target}** = desired target class
- **f(x)** = model prediction

After each update, the perturbation is projected back into an **L_∞ constraint region** to ensure that the modification remains

small. **The L_∞ constraint region** limits how much each pixel of the image can change during the adversarial attack.

$$\| x_{adv} - x \|_\infty \leq \varepsilon$$

where:

- **x_{adv}** = adversarial image
- **x** = original image
- **ε** = maximum allowed perturbation

This constraint ensures that the adversarial image remains visually similar to the original image.

<u>Hyperparameters Used:</u>

Parameter	Value	Description
Target class	3	Forces digit 7 -> 3
Epsilon (ε)	0.05	Maximum allowed perturbation
Step size(α)	0.005	Magnitude update per iteration
steps	40	Maximum attack iteration
Success threshold	0.9	Targets confidence threshold

Table 5: Hyperparameters

These parameters ensure the targeted execution.

Execution:

- A set of **10 images of digit 7** were selected from the test dataset.
- The PGD attack was applied to both models:
  - **Lazy Model** (trained without intervention)
  - **Robust Model** (Version 2: Color Randomization + Channel Dropout)
- The attack attempts to convert the prediction from **7 to 3**.
- The following metrics were recorded:
  - Number of attack steps required
  - Maximum perturbation magnitude ($L\infty$ distance)
- It was executed on both 5 epoch and 10 epoch models.

Results (5 epoch model):

Single example Attack:

Lazy model confidence: 0.2117

Robust model confidence: 0.9497

Surprisingly, Attack successfully fooled the robust model

$L\infty$ Perturbation for This Image:

Lazy Model 0.05
Robust Model: 0.05

It means both attacks used maximum allowed perturbation(ε=0.05).

Average Result(Across 10 images):
Avg steps (lazy): 39.0
Avg steps (robust): 12.8
Avg L_inf (lazy): 0.05
Avg L_inf (robust): 0.039

Upon interpretation, we can say that the **robust** model was easier to fool with respect to the **lazy** model after the targeted PGD attack. **Robust** model despite improving generalisation across images fall prey to PGD attack. One likely reason is that it was not robust to small pixel-level gradient. As for **lazy** model, the prediction game was unstable and hence it couldn't fool an already weak model.

<u>Results (10 epoch model):</u>

Single example Attack:

Lazy model confidence: 0.68

Robust model confidence: 0.38

Neither model was successfully fooled.

L_∞ Perturbation for This Image:

Lazy Model 0.05
Robust Model: 0.05

It means both attacks used maximum allowed perturbation(ε=0.05).

Average Result(Across 10 images):
Avg steps (lazy): 31.4
Avg steps (robust): 25.5
Avg L_inf (lazy): 0.047
Avg L_inf (robust): 0.0425

Upon interpretation of the 10 epoch model with respect to the 5 epoch model, we can say that on average that attack took 31.4 iterations to fool the lazy model and 25.5 iterations for the robust model. The average perturbation was close to full value of 0.05 for both the cases. The 10-epoch model showed improved resistance to adversarial attack, particularly for the robust model where the number of attack iterations increased substantially.

Conclusion:

The experiment shows that while the intervention model improves generalization by reducing color bias, adversarial robustness depends strongly on training duration. With longer training, the robust model became more resistant to adversarial attack.

Task 6: Sparse Autoencoder

Introduction:

A Sparse Autoencoder(SAE) is a specific type of unsupervised neural designed to find “Hidden” patterns in data while keeping most of its internal “brain cells” turned off.
Autoencoders try to compress data into a smaller space. The SAE does the opposite: It often expands the data into a larger space, but with a strict rule: only few neurons are allowed to be active at once.

- **The Input:** Messy, "polysemantic" activations (where one neuron might represent many different things).
- **The Bottleneck (Hidden Layer):** A massive "dictionary" of potential features, where 99% of them are forced to be zero.
- **The Output:** A reconstruction of the original input.

Objective:

A Sparse Autoencoder was trained to understand internal representation of the models learned on final convolutional activations of both the 5 epoch and 10 epoch models. The goal is to decompose hidden states into sparse interpretable components.

Execution:

Approximately 5,000 hidden activations were extracted from the last convolutional layer (before classification), resulting in

spatial dimension of 128*7*7, which were flattened feature vectors of dimension 6272 per image. In SAE the features were increased twice the dimension i.e. 12,544. A sparse autoencoder was trained to reconstruct these activations using a reconstruction loss combined with an L1 sparsity penalty on the latent code.

To analyze these representations, we trained a **Sparse Autoencoder** consisting of:

- Encoder: Linear layer mapping 6272 → hidden latent space
- ReLU activation to enforce non-negativity
- Decoder: Linear layer reconstructing the original 6272-dimensional activation

The SAE was trained using a combination of **reconstruction loss and sparsity regularization**:

$$\text{Loss} = \text{MSE}_{(\textbf{reconstruction})} + \text{Lambda}|h1|$$

where **h** represents latent activation vector.

After training, the variance was computed of each latent neuron across the dataset and selected high-variance neurons for interpretability analysis. For each selected neuron, the top activating images were visualised and examined for both digit labels and color assignments.

To assess causality, Targeted interventions were performed: for a selected latent neuron, we “dialed down” its activation (set it to zero), decoded the modified representation, and passed it through the original classifier to observe prediction changes.

Results – 5 Epoch Model

The 5-epoch model exhibited strong shortcut entanglement. High-variance SAE neurons are consistently activated on pink digit “5” images and blue “2” images. These neurons encoded a conjunction of digit identity and color rather than shape. The top 10 neurons that were activated are [6119, 2594, 5427, 1546, 3874, 3744, 10434, 5289, 9474, 4651].

When we intervened on one such neuron (e.g., neuron 6119) by setting its activation to zero, the model’s prediction changed from 5 to 2. This demonstrates that the model relied heavily on shortcut-aligned/color-bias internal features. The representation was sparse and fragile, with individual neurons exerting strong causal influence over classification.

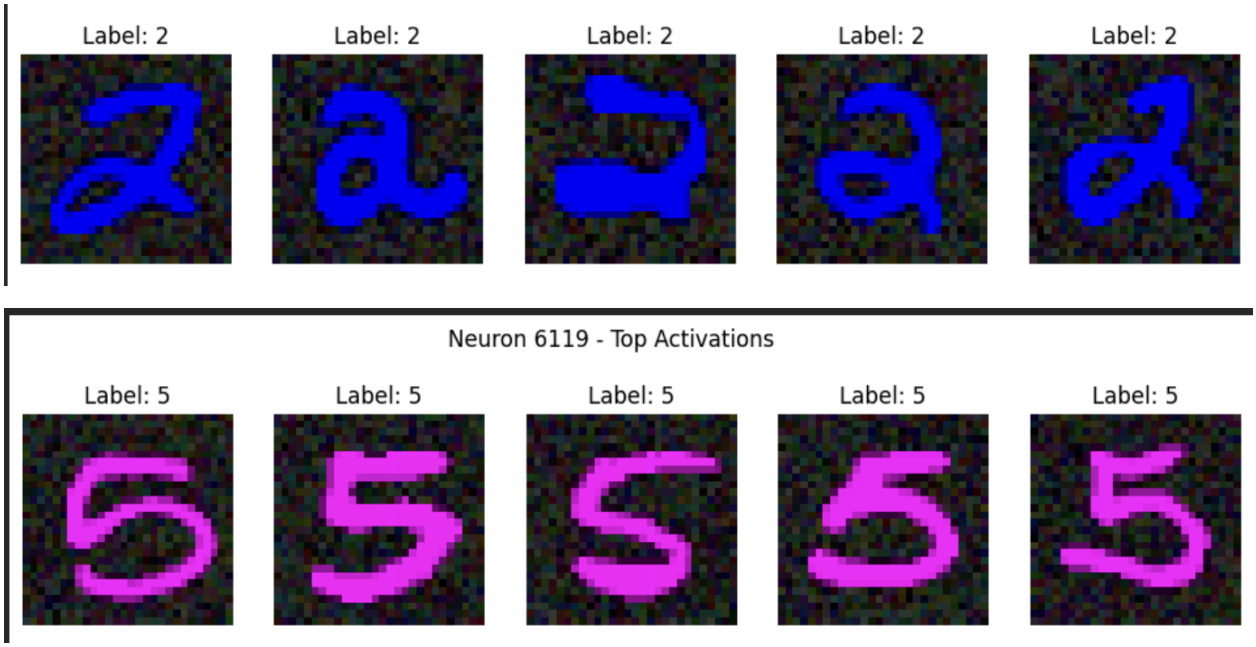

Fig. 5 epoch results.

Results – 10 Epoch Model

The 10-epoch model displayed more structured internal representations. Several high-variance neurons activated selectively for specific digit shapes (e.g.,0,4,5,8,9), suggesting improved shape-based encoding compared to the earlier model. It also picked up same color digits as evident from the pictures. The top 10 neurons that were activated are [4664, 1076, 12336, 5596, 6160, 445, 8727, 8162, 403, 3822].

However, some neurons (e.g., neuron 4664) still activated primarily on pink 5 images, indicating that shortcut-aligned features persisted in the hidden state. Intervening on such a neuron altered predictions from 5 to 9, confirming that these features retained causal influence.

Hence, while extended training improved representational quality and introduced more digit-specific features, shortcut traces were not fully eliminated.

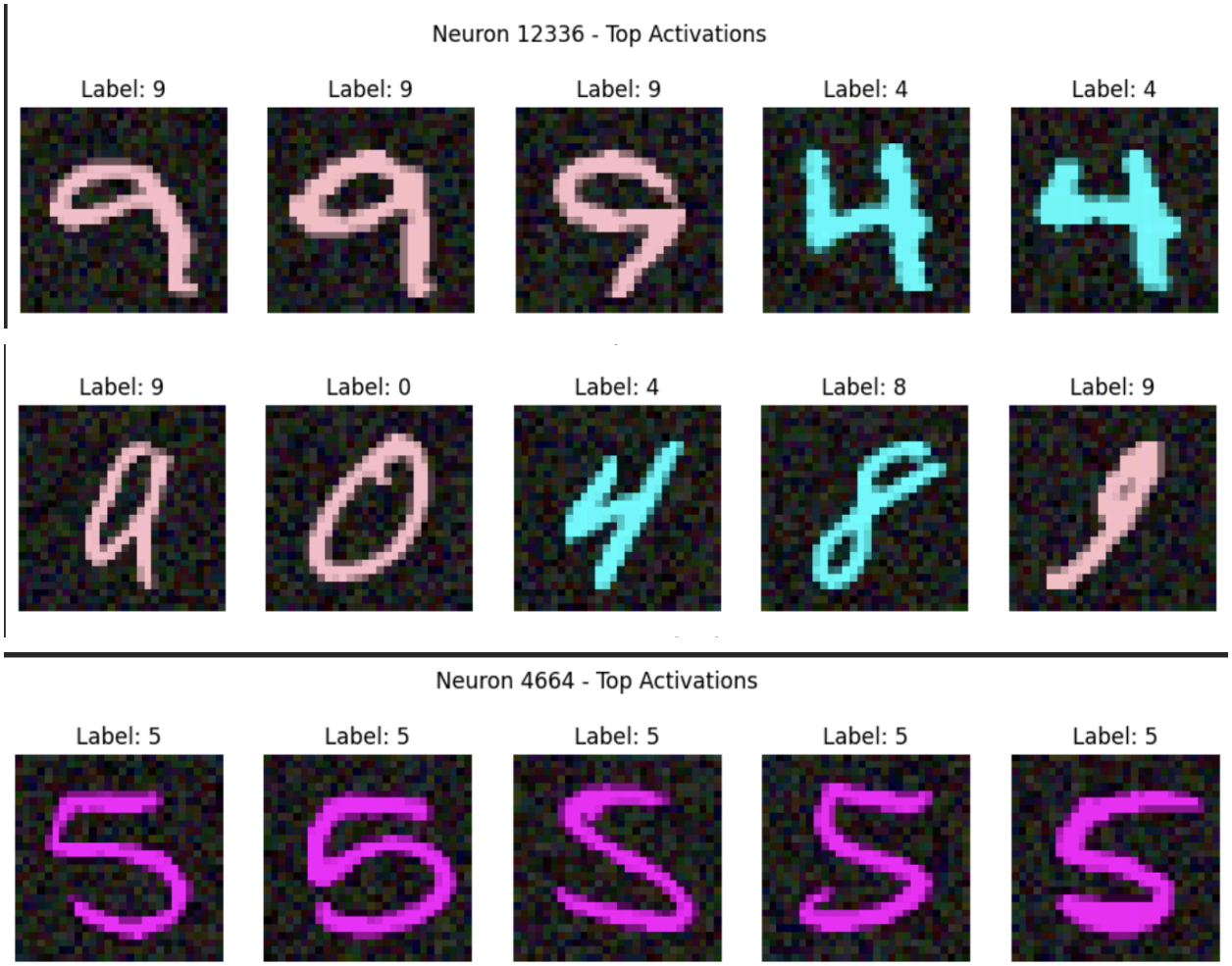

Fig. 10 epoch Results

Conclusion

Sparse Autoencoder decomposition revealed that shortcut learning manifests as entangled color-digit neurons in early training. With more training i.e. 10 epoch, the model develops

stronger digit oriented or shape representations, but shortcut-aligned features may persist within the hidden state.

Causal interventions demonstrate that specific latent features significantly influence predictions, highlighting the usefulness of SAE-based analysis for mechanistic interpretability. Overall, the 10-epoch model is less shortcut-dominated than the 5-epoch model, but not fully disentangled.

Learning Outcome:

This project showcased how neural networks can learn shortcut signals when strong correlation exists in the training data. SimpleCNN achieved high training accuracy in the color -biased MNIST dataset but failed to generalise on the Hard Test set which was made sure that it wasn't biased. It relied on Colors rather than shape. Interpretability techniques like activation maximization and Grad-CAM revealed that it relied mostly on color than shape. Bias mitigation techniques/strategies like color randomization and channel dropout significantly improved generalization by improving shape-based learning. However, during adversarial attack, even improved models fall prey to the gradient based perturbations. Sparse Autoencoder analysis further demonstrated that early models encoded entangled color-digit features, while longer training produced more shape oriented internal representations.

This study highlights the importance of dataset design, interpretability, and robustness evaluation in developing robust, interpretable and reliable ML models.

References:

1) PyTorch documentation.
2) IIT Madras Deep Learning(CS7015): Lec 7.5 Sparse Autoencoders(Youtube)
3) ChatGPT and Gemini(for learning assistance and debugging purpose)
4) Neuronpedia
5) Selvaraju, R. R., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks. ICCV.
6) Madry, A., et al. (2018). Towards Deep Learning Models Resistant to Adversarial Attacks. ICLR.